Advanced Typing

CS 1622

Jarrett Billingsley

Class Announcements

- the exam is on Monday, don't forget!
  - we'll do a little review before we start.
  - **it will be taken *on Canvas,* but *in person,*** so bring a computer.
  - also **do not use Safari,** mac users! issues with Canvas exams!
- now let's do some ADVANCED TYPING



Polymorphism

Poly = Many, Morph = Shape

- **abstraction** is about ignoring concrete details.
- **polymorphism** lets us write abstract code **once,** that can work on **many types of values,** without having to change the code.
- it covers a *wide* range of techniques and language features, like…
  - **subclasses** in Java, C++, Python
  - **interfaces** in Java, Go and **traits** in Rust
  - **generics** in Java, Rust
  - **templates** in C++
  - **function name overloading** in Java, C++
  - **operator overloading** in C++, Rust, Python
  - **unions** and **void pointers** in C
  - **duck typing** in most dynamically-typed languages
  - anything with **"late binding"** in the name

Don't repeat yourself.

- repeating code is a *cardinal sin.* **encode your thoughts once.**
- polymorphism lets us avoid repeating code.

```
ArrayList<Integer> i = new ArrayList<>();
ArrayList<String>  s = new ArrayList<>();
```

the task of keeping an array-based list of things doesn't change depending on what type of things are in the array.

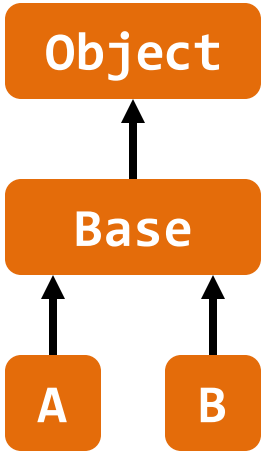
we don't have to make a new **`ArrayList`** class for every type of value we want to store. we **wrote it once,** and now it will work for any type of value.

we can even make functions which operate on *any type* of **`ArrayList`**:

```
<T> void foo(ArrayList<T> list)
```

What type is it?

- the basic idea is that polymorphism lets us make values which can have **more than one possible type.**



with this class hierarchy, I can make a **Base** variable which can hold 3 different types.

```
void print(int i);
void print(String s);
```

what type is this **print** method?

I guess it's a set of types...?

"Ad-Hoc" Polymorphism

You know about this: overloading!

- this is something you can do in Java (*but not Rust*):

```
void print(int i);
void print(String s);
```

- now when I call **`print(x)`...**
  - which version, or **overload,** is used is decided **statically based on the type of the argument.**
- each overload can do **different things,** which can be useful!
  - ...but if they're all doing basically the same thing, it feels silly.
  - have a look at **java.util.Arrays** for an example of what I mean.
- it's called **ad-hoc** because you just add overloads as you need them, and the compiler figures out what you mean.
- so how would this be handled **by a compiler?**

Implementing overloading in a compiler

- essentially, we're just extending the symbol table.

Name	Referent
`"A"`	`<class A>`
`"print"`	*`<overload set>`*



Signature	Referent
`void(int)`	`<print 1>`
`void(char)`	`<print 2>`
`void(String)`	`<print 3>`

as the user adds more overloads, the set can be extended.

now a name can refer to either a **single** symbol or a **set** of symbols with the same name.

now, when **print** is called, it searches for the **right signature** in the overload set, and gives an error if **none** match or **more than one** match.

More than one can match??

- overloading is complicated by **implicit type conversions.**
- for example, Java will implicitly convert ints to floats.

```
static void print(int i, float f)...
static void print(float f, int i)...
print(10, 10);
```

`10` can be implicitly converted to float, so this use of **print** could mean *either* overload. the compiler says this is **ambiguous.**

you don't even wanna know how ugly this gets in languages like C++ where you can define *your own implicit conversions*.

Name mangling

- for practical reasons (linkers, debuggers, etc), these overloaded functions must be given **unique names** in the output object files.
- **name mangling** encodes the function's **signature –** its argument and return types – into a textual form that these other tools can use.

```
void print(int i, float f)
 => "_Z5printif"        (g++)
 => "?print@@YAXHM@Z" (msvc++)
void print(float f, int i)
 => "_Z5printfi"        (g++)
 => "?print@@YAXMH@Z" (msvc++)
```

now you know what these horrid things are, if they ever show up in error messages…

Subtype Polymorphism

You know about this one too!

- in subtype polymorphism, types can be placed in a **hierarchy,** where subtypes are *more specialized* than their base types.
- the important feature is: **anywhere a base type is expected, a subtype can be used in its place.**

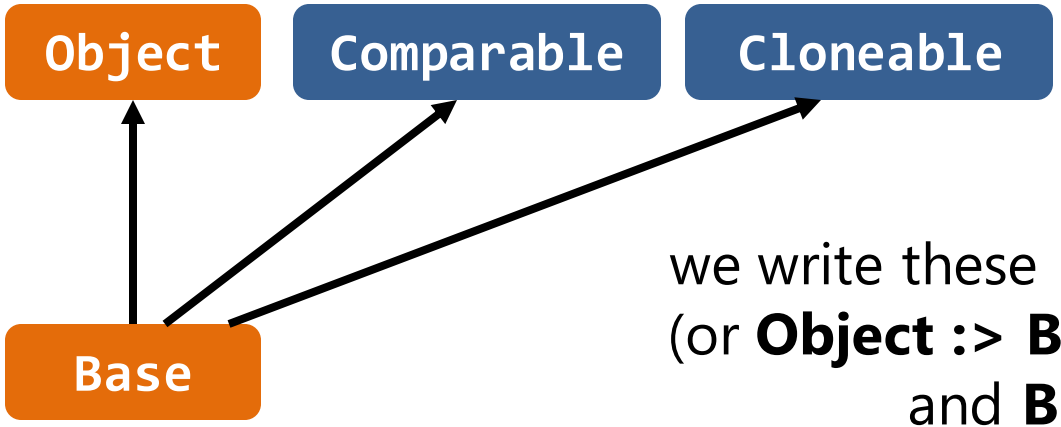
```
void print(Object o) {
  System.out.println(o.toString());
}
```

any object type can be passed to this function, because **all** object types are, directly or transitively, subtypes of **`Object`**.

another way of thinking of it is that the **set of operations** of each object type is a *superset* of the set of operations of **`Object`**.

Interfaces

- Java also has **interfaces.** every type can extend *one* class but can implement *any number* of interfaces.



now Base is a subtype of *three other types*.

we write these relations as **Base <: Object** (or **Object :> Base**), **Base <: Comparable**, and **Base <: Cloneable**.

but interfaces exist "outside" the class hierarchy, letting us group types by their **capabilities,** not just their inheritance.

Implementing subtyping in a compiler

- the compiler has to keep track of the subtype relations between classes and interfaces.
  - it also has to ensure that there are **no cycles** in the subtypes, like:

```
class A extends B {}
class B extends A {} // how??
```

- then when doing typechecking, it will allow any type *or its subtype* to be put into a variable, argument, array slot etc.
  - `Object o = new A();` is valid because **A** is a subtype of **Object**
  - `A a = new Object();` is *not,* because the opposite is not true.
- it really is that straightforward!
  - …. ha ha ha sure yeah right

But it's got... issues.

- subtyping comes at a cost: **complexity.**
- it interacts with every other feature of a language, often in non-trivial ways, and even making some type systems **undecidable.**

```
void print(Object o)...
void print(A a)...

A a = new A();
// calls print(A)
print(a);
Object o = a;
// calls print(Object)
print(o);
```

does this make sense? is this what you'd expect?

this happens because Java's **ad-hoc** polymorphism is ***statically*** *resolved,* but its **subtype** polymorphism is ***dynamically*** *resolved.*

other languages have other ideas about this – look up *multiple dispatch.*

Implementing subtyping at runtime

- we won't get into the details for a few weeks, but…
- each class gets a **data layout** and a **virtual method table (vtable).**
- subclasses' layouts and vtables are **prefixed** by their superclasses'.

```
class Object
```

layout
vtable
monitor

vtable
<Object>
clone()
equals()
toString()

```
class A { int x; void foo() {} }
```

layout
vtable
monitor
x

vtable
<A>
clone()
equals()
toString()
foo()

any A reference therefore points to a piece of memory which looks identical to an Object at the beginning.

Subtyping in Rust

- Rust has no OOP, but it *does* support a form of subtyping and dynamic dispatch using **traits,** which are similar to Java interfaces.

```
trait Base {
  fn base(&self);
}
// Derived is called a
// supertrait of Base
trait Derived: Base {
  fn derived(&self) {
    // we can use
    // methods of Base
    self.base();
  }
}
```

now any type which implements **Derived** must also implement **Base.**

you can also have variables of type **Box<dyn Base>**, which are like Java's interface variables.

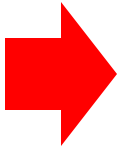
they can hold *any* type which implements **Base**, and their methods will be dispatched at **runtime.**

Parametric Polymorphism

Static and Dynamic

- **ad-hoc** polymorphism is statically dispatched.
- **subtype** polymorphism is (mostly) dynamically dispatched.
- but dynamic dispatch is actually **kind of costly** on modern CPUs…

```
o.toString();
```



```
lw a0, 4(sp)   # a0 = o
lw t0, 0(a0)   # t0 = o.vtbl
lw t0, 16(t0)  # t0 = o.vtbl[4]
jalr t0        # indirect call
```

three **dependent** loads from *three* totally different places in memory make the pipeline and cache *very* unhappy.

and then you have an **indirect jump,** which can confuse the branch predictor and stall the instruction pipeline!

Parametric polymorphism to the rescue

- **parametric polymorphism** is a more **explicit** kind of polymorphism, where the types are a sort of "fill in the blank."
- the types themselves *become an argument*.

```
fn id<T>(t: T) -> T {
  return t;
}
```

this is a silly function which just returns its argument. "id" means "identity." it's math stuff.

this is not *one function*. this is **an entire family of functions,** which take a **T** and return a **T**, **for *all* types T.**

this is **universal quantification.**
we're saying **∀T,** you can get an id function of type **T→T**
(a function that maps from T to T).

what "type" is id, then?

Using parametrically polymorphic functions

- Rust lets you either explicitly or *implicitly* specify the type arguments.

```
let x: i32 = id::<i32>(10);
let y: i32 = id(20);
let z = id(30);
```

the `::<>` operator is called "turbofish." 

in the second line, the compiler is able to **infer** what type should be passed as the type argument.

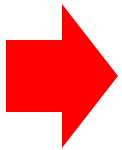
in the third line, it infers **the type of the variable** too!

all three lines use the **same implementation** of id, `id::<i32>`.

Monomorphization

- but CPUs don't understand polymorphic code.
- **monomorphization** converts each use of a polymorphic function into a concrete, **monomorphic** ("one-shape") version of itself.

```
let x = id(10i32);
let y = id(3.5f64);
let z = id("hello");
```



```
id::<i32>
id::<f64>
id::<&str>
```

the compiler looks at **every use of the function across the entire program** to determine the implementations needed.

it will **reuse** implementations if used in multiple locations (like if `id::<i32>` is used 100 times).

essentially, **it's automatically generating all the ad-hoc overloads for us!**

Generics without monomorphization

- monomorphization gives you the **best possible performance...**
  - but it can slow compilation, and produce a larger executable.
- so, **in Java,** generics are **not** monomorphized!
  - if you have **`ArrayList<Integer>`** and **`ArrayList<String>`**, the *compiler* treats them as separate types, but...
  - both of those use **the exact same code at runtime.**
  - under the hood, all versions of **`ArrayList`** just use **`Object`** variables and downcasts and **`instanceof`** to make it work.
- this has advantages in compilation time and code size... but it comes at two big costs.
  - one, **you cannot use non-object types in Java generics.**
    - this is why **`ArrayList<int>`** is invalid!
  - two, **the performance is mediocre.**
    - but that's not really a main goal for Java so whatever!

Bounded Quantification

(or, "making Parametric Polymorphism actually useful")

∀ only gets you so far.

- remember, a type has a set of **values** and a set of **operations.**
- what **operations** can you do with a variable that can be ***any*** *type?*
  - can you add it? index it? even print it out?
  - well... no. **because you don't know if the type will support those operations.**
  - about all you *can* do is copy values around (put them in variables, pass them as arguments, return them) and that's not too useful.
- well if we can make **universal** types, what about **∃ existential types?**
- an **existential type** (or **bounded quantification**) gives you the flexibility to accept multiple types...
  - while saying, "they need to support **at least** this set of operations."
- it's a little easier to demonstrate than explain.

Bounded quantification in Java

- remember **extends** in generics?

```
<T extends Comparable<T>> T max(T a, T b) {
  return (a.compareTo(b) > 0) ? a : b;
}
```

this works for any type T ***as long as*** that type implements the **Comparable** interface (against other values of the same type).

```
System.out.println(max(3, 4));      // prints 4
System.out.println(max("a", "b")); // prints b
```

without the **extends Comparable<T>**, we wouldn't be able to call **.compareTo()** in the method.

Bounded quantification in Rust

- Rust's **traits** do the same job as Java's **interfaces** here.

```
fn max<T: PartialOrd>(a: T, b: T) -> T {
    if a > b { a } else { b }
}
```

`PartialOrd` is the closest thing Rust has to Java's **Comparable**, and the resulting function can be used the same way:

```
println!("{}", max(3, 4));     // prints 4
println!("{}", max(3.3, 4.4)); // prints 4.4
```

But what about using subtype polymorphism?

- couldn't we just make our Java function take two Comparables?
- well yes, *but*...

```
Comparable max(Comparable a, Comparable b) {
  return (a.compareTo(b) > 0) ? a : b;
}
```

now *this* nonsense code compiles and runs, and we get a **runtime error** instead:

```
System.out.println(max(3, "a"));
```

parametric polymorphism **moves the type checking into the compiler,** allowing us to catch errors *without* having to run the code, and *without* the performance penalty of runtime type-checking*.

Summary

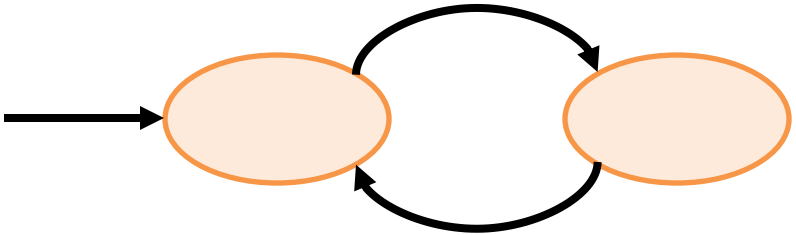
The kinds of polymorphism

- **ad-hoc** polymorphism is also called **function overloading.**
  - it lets you define multiple versions of the same function with different type signatures.
  - the version that will be used is selected **at compile time.**
- **parametric** polymorphism is also called **generics** or **templates.**
  - it lets you define "fill-in-the-blanks" classes, structs, functions etc.
  - the blanks can be filled in with *any* type… or with bounded quantification, only types which *satisfy some condition*.
  - the types are filled in and checked **at compile time.**
  - the generated code *may* or *may not* be **monomorphized.**
- finally, **subtype** polymorphism is a feature of classes and interfaces.
  - a **derived** value can be used anywhere a base type is required.
  - when a **virtual** method is called, the version that is used is selected **at runtime,** based on the type of the object.

The limits of typing and type systems

- it doesn't matter how fancy your type system is…
  - **it will never allow *all* correct programs.**

we saw this previously when talking about Box in Rust.



this seemingly-simple object graph is *impossible to directly represent* in Rust because of its type system.

but that's the *point*. **type systems limit what we can do, because unlimited ability leads to mistakes.**

Coloring inside the lines

- type systems force us to *work within a set of limitations,* which can sometimes be annoying…
  - but it can also lead to **simpler, more elegant solutions.**
- because static type systems are automatically checked by the compiler, **playing by their rules can avoid many silly mistakes.**
  - programming is *hard*. you need all the help you can get!
- and with that…
  - we're done with the material before the exam ö